

» Algorithms For Training ML Models

All of the main ML libraries provide several choices of optimization algorithm for training models. Why? What's the difference between them, and how to choose?

TensorFlow/Keras:

```
model.compile(  
    optimizer=keras.optimizers.RMSprop(), # Optimizer  
    # Loss function to minimize  
    loss=keras.losses.SparseCategoricalCrossentropy(),  
    # List of metrics to monitor  
    metrics=[keras.metrics.SparseCategoricalAccuracy()],  
)
```

Available optimizers

- SGD
- RMSprop
- Adam
- Adadelta
- Adagrad
- Adamax
- Nadam
- Ftrl

» Algorithms For Training ML Models

PyTorch:

Algorithms

Adadelta

Implements Adadelta algorithm.

Adagrad

Implements Adagrad algorithm.

Adam

Implements Adam algorithm.

AdamW

Implements AdamW algorithm.

SparseAdam

Implements lazy version of Adam algorithm suitable for sparse tensors.

Adamax

Implements Adamax algorithm (a variant of Adam based on infinity norm).

ASGD

Implements Averaged Stochastic Gradient Descent.

LBFGS

Implements L-BFGS algorithm, heavily inspired by [minFunc](#).

RMSprop

Implements RMSprop algorithm.

Rprop

Implements the resilient backpropagation algorithm.

SGD

Implements stochastic gradient descent (optionally with momentum).

» Algorithms For Training ML Models

sklearn:

sklearn.neural_network.MLPClassifier

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=100, activation='relu', *, solver='adam', alpha=0.0001,
batch_size='auto', learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200, shuffle=True,
random_state=None, tol=0.0001, verbose=False, warm_start=False, momentum=0.9, nesterovs_momentum=True,
early_stopping=False, validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_no_change=10,
max_fun=15000)
```

[\[source\]](#)

solver : {'lbfgs', 'sgd', 'adam'}, default='adam'

The solver for weight optimization.

- 'lbfgs' is an optimizer in the family of quasi-Newton methods.
- 'sgd' refers to stochastic gradient descent.
- 'adam' refers to a stochastic gradient-based optimizer proposed by Kingma, Diederik, and Jimmy Ba

» Algorithms For Training ML Models

The main algorithms popular are:

- * SGD - Stochastic Gradient Descent
- * Adam - Adaptive Moment Estimation
- * RMSProp — Root Mean Square Propagation.
- * Adagrad - Adaptive gradient Descent.
- * L-BFGS - BFGS = Broyden-Fletcher-Goldfarb-Shannon algo,
L=limited memory. Only suitable for smaller learning tasks

Let's look more closely at these, plus some other algos for context.

» Basic Optimisation Algorithm

* Basic structure of an optimisation algorithm to minimise function $f(x)$ is:

- ✓ (i) initialise x
- ✓ (ii) calculate step i.e. change to x
- ✓ (iii) $x = x - \text{step}$
- ✓ (iv) repeat (ii)-(iii) until result is good enough or run out of time

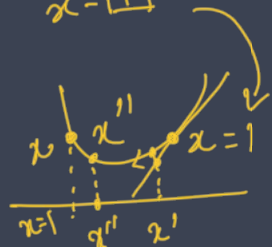
In python:

```
x=x0  
for k in range(num_iters):  
    step = calcStep(fn,x)  
    x = x - step
```

* How to calculate step?



$x - \boxed{+}$



$$y = \textcircled{m}x + c$$

↓
Gradient

$$x - \boxed{-} = x + \boxed{+}$$

$$Q \leq 0$$

$$Q \geq 0$$

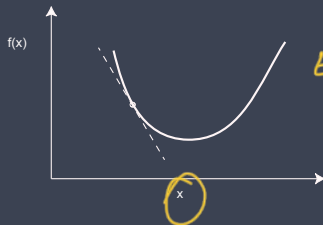
» Derivatives

- * When x is scalar the *derivative* of a function $f(\cdot)$ at point x is the *slope* of the line that just touches the function at x e.g.

$$y = \textcircled{m}x + c$$

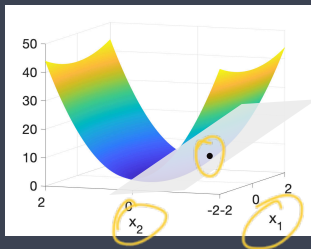
$\downarrow$

$$\frac{dy}{dx}$$



← graph Quadratic
 $\parallel \frac{d}{dx}(\text{Quad.}) =$
 linear
 $\downarrow$
 line

- * When $x = [x_1, x_2]$ then the derivative is the *slope* of the plane that just touches the function at x e.g.



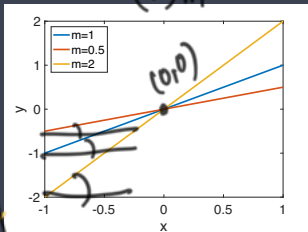
← Surface
 $\frac{\partial}{\partial x_1, x_2}(\text{Surface}) = \text{plane}$

- * And similarly when x has > 2 elements, but can't draw it.

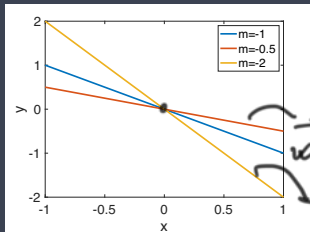
» Derivatives

- * Equation of a line is $y = mx + c$. m is slope, c is intercept (when $x = 0$ then $y = c$).

(+)m

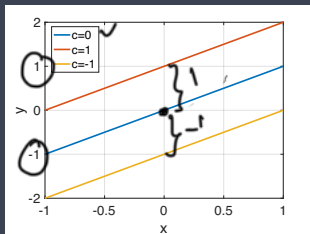


✓ $c=0$



(-)m

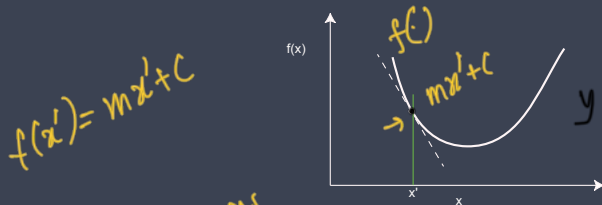
✓ $y = -0.5x$
✓ $y = -x$
✓ $y = 2x$



$m=1$ $c=0$
 $y = mx + c$
 $m=1$ $y = x + c$
 $m=0.5$ $y = 0.5x + c$
 $m=2$ $y = 2x + c$

» Derivatives

- * *Derivative* of a function $f(\cdot)$ at point x' is the slope of the line that just touches the function at x'



$$y = mx + c$$

$$y = x^2$$

$$f(x) = x^2$$

$$f'(x) = 2x$$

$$f(x) =$$

- * Equation of a line is $mx + c$, slope m and intercept c .
- * Line touches $f(\cdot)$ at point x' , so $mx' + c = f(x')$ i.e. $c = f(x') - mx'$ ✓
- * Slope m of line is derivative, i.e. $m = \frac{df}{dx}(x)$ (this is just notation¹, the important point is that we can calculate $\frac{df}{dx}(x)$ and so m using standard tools).
- * Putting this together, equation of line is

$$\frac{df}{dx}(x')(x - x') + f(x')$$

and so

$$\Rightarrow f(x) \approx f(x') + \frac{df}{dx}(x')(x - x')$$

$$mx + c$$

$$\downarrow \frac{df}{dx}(x')x + f(x')$$

$$- \frac{df}{dx}(x')x'$$

¹ sometimes $f'(x)$ is used instead of $\frac{df}{dx}(x)$

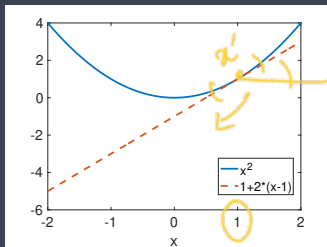
» Derivatives:

Example:

* $f(x) = x^2$. Then $\frac{df}{dx}(x) = 2x$

* At point $x' = 1$,

$$f(x) \approx f(1) + \frac{df}{dx}(1)(x - 1) = 1 + 2(x - 1)$$



» Derivatives:

Example:

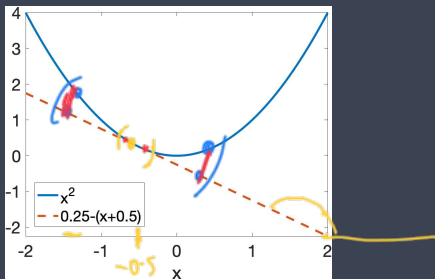
* $f(x) = x^2$. Then $\frac{df}{dx}(x) = 2x$

* At point $x' = -0.5$, $\checkmark$

$$\frac{df}{dx} \leq 0$$

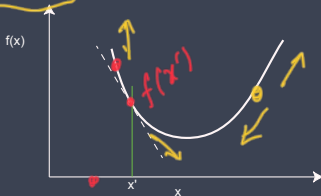
$$\begin{aligned} f(x) &\approx f(-0.5) + \frac{df}{dx}(-0.5)(x - (-0.5)) = 0.5^2 + 2(-0.5)(x - (-0.5)) \\ &= 0.25 - (x + 0.5) \end{aligned}$$

$$0.25$$



» Derivatives

$$* f(x) \approx f(x') + \frac{df}{dx}(x')(x - x')$$



* Suppose we choose $x = x' - \alpha \frac{df}{dx}(x')$, then (2)

$$\begin{aligned} (3) \rightarrow f(x) &\approx f(x') + \frac{df}{dx}(x')(x' - \alpha \frac{df}{dx}(x') - x') \\ &= f(x') - \alpha \left(\frac{df}{dx}(x') \right)^2 \end{aligned}$$

* The square of a number is always ≥ 0 , so $\left(\frac{df}{dx}(x') \right)^2 \geq 0$. And so

$$f(x) \lesssim f(x')$$

i.e. moving from point x' to $x = x' - \alpha \frac{df}{dx}(x')$ tends to decrease function $f(\cdot)$.

* Only approximate though, because line is only an approximation to function $f(\cdot)$ near point x'

» Derivatives

$(x, f(x))$

- * Equation of a plane is $y = m_1 x_1 + m_2 x_2 + c$
- * Note: we now have two slopes m_1, m_2 and intercept c
- * Notation: For plane just touching $f(\cdot)$ at point x'
 - * $m_1 = \frac{\partial f}{\partial x_1}(x'), m_2 = \frac{\partial f}{\partial x_2}(x')$
 - * $\frac{\partial f}{\partial x_1}(x')$ is the *partial derivative* of $f(\cdot)$ wrt x_1 at point x'
 - * $\frac{\partial f}{\partial x_2}(x')$ is the *partial derivative* of $f(\cdot)$ wrt x_2 at point x'
 - * $\nabla f(x') = [\frac{\partial f}{\partial x_1}(x'), \frac{\partial f}{\partial x_2}(x'), \dots, \frac{\partial f}{\partial x_n}(x')]$, the vector of partial derivatives. And sometimes $\nabla_{x_1} f(x')$ is used for $\frac{\partial f}{\partial x_1}(x')$ etc.
- * This plane is an approximation to function $f(\cdot)$ near point x' i.e.

$$f(x) \approx f(x') + \frac{\partial f}{\partial x_1}(x')(x_1 - x'_1) + \frac{\partial f}{\partial x_2}(x')(x_2 - x'_2)$$

- * If we choose $x_1 = x' - \alpha \frac{\partial f}{\partial x_1}$ and $x_2 = x' - \alpha \frac{\partial f}{\partial x_2}$ then moving from point x' to x tends to decrease function $f(\cdot)$ i.e.
 $f(x) \lesssim f(x')$

» Derivatives

- * In general, when vector x has n elements then

$$f(x) \approx f(x') + \frac{\partial f}{\partial x_1}(x')(x_1 - x'_1) + \frac{\partial f}{\partial x_2}(x')(x_2 - x'_2) + \dots + \frac{\partial f}{\partial x_n}(x')(x_n - x'_n)$$

- * Choosing x with $x_1 = x' - \alpha \frac{\partial f}{\partial x_1}$, $x_2 = x' - \alpha \frac{\partial f}{\partial x_2}$, $\dots$,
 $x_n = x' - \alpha \frac{\partial f}{\partial x_n}$ then moving from point x' to x tends to decrease function $f(\cdot)$

» Calculating Derivatives

- * Derivative of $f(x) = x^2$:

```
import sympy
x = sympy.symbols('x', real=True)
f=x**2
dfdx = sympy.diff(f,x)
print(f,dfdx)
```

Output is:

x^2 $2*x$

- * Derivative of $f(x) = 0.5(x_0^2 + 10x_1^2)$, vector $x = [x_0, x_1]$:

```
import sympy
x0, x1 = sympy.symbols('x0, x1', real=True)
x=sympy.Array([x0,x1])
f=0.5*(x[0]**2+10*x[1]**2)
dfdc = sympy.diff(f,x)
print(f,dfdx)
```

Output is:

$0.5*x_0^2 + 5.0*x_1^2$ $[1.0*x_0, 10.0*x_1]$

- * Also tensorflow <https://www.tensorflow.org/guide/autodiff>, pytorch https://pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html

» Verifying Derivative Calculations

Can use *finite difference* approximation to a derivative as a sanity check.:

* Recall

$$f(x) \approx f(x') + \frac{\partial f}{\partial x_1}(x')(x_1 - x'_1) + \frac{\partial f}{\partial x_2}(x')(x_2 - x'_2) + \dots + \frac{\partial f}{\partial x_n}(x')(x_n - x'_n)$$

* Select $x = x'$ and then add a small amount δ to element 1 of x
i.e.

$$x = [x'_1 + \delta, x'_2, \dots, x'_n]$$

Then

$$f(x) \approx f(x') + \frac{\partial f}{\partial x_1}(x')\delta$$

i.e.

$$\frac{\partial f}{\partial x_1}(x') \approx \frac{f(x) - f(x')}{\delta}$$

* Perturbation δ needs to be small e.g. 0.01 or less.

» Verifying Derivative Calculations

Example:

- * $f(x) = x^2$, $\frac{df}{dx}(x) = 2x$
- * At point $x' = 1$ then $\frac{df}{dx}(1) = 2.0$
- * Finite difference:

$$\frac{f(x' + \delta) - f(x')}{\delta} = \frac{f(1 + \delta) - f(1)}{\delta} = \frac{(1 + \delta)^2 - 1}{\delta}$$

For $\delta = 0.1 \rightarrow 2.1$

For $\delta = 0.01 \rightarrow 2.01$

For $\delta = 0.001 \rightarrow 2.0010$

» Verifying Derivative Calculations

* Derivative of $f(x) = x^2$:

```
import sympy
x = sympy.symbols('x', real=True)
f=x**2
dfdx = sympy.diff(f,x)
f = sympy.lambdify(x, f)
dfdx = sympy.lambdify(x, dfdx)
x=np.array([1.0])
print(dfdx(*x))
delta=0.01
print( ((x+delta)**2 - x**2)/delta)
```

Output is:

2.0

2.01

» Gradient Descent

```
x=x0  
for k in range(num_iters):  
    step = calcStep(fn,x)  
    x = x - step
```

Now we know one way to choose the step, namely:

$$step = \alpha \left[\frac{\partial f}{\partial x_1}(x), \frac{\partial f}{\partial x_2}(x), \dots, \frac{\partial f}{\partial x_n}(x) \right]$$

where α is the *step size* or *learning rate*